PS No. 97 | Python Data Science

PROJECT REPORT:
STUDENT PERFORMANCE ANALYZER

Python 3 | CSV Data Processing | Validation | Statistics | Data Visualization

Name:	**Yukti Sahu**	Course:	**BCA AI/ML (Association with Google)**
ERP ID:	**RU-26-01699**	Section:	**B**
University:	**Rungta International Skills University**	Submitted To:	**Satyaprakash Sir**

PROJECT FOUNDATION

1. Project Overview & Objectives

CSV Data Ingestion

Reads student performance records dynamically from `students.csv`. Validates entries and filters out rows containing invalid numeric marks.

Input Processing

Automated Grade & Stats

Computes total marks out of 300, percentage, and letter grades (A, B, C, D, F). Calculates class mean, median, standard deviation, and topper.

Core Calculations

Data Visualization

Generates graphical visualizations using Matplotlib including subject-wise average bar charts and percentage distribution histograms.

Matplotlib Charts

IMPLEMENTATION CODE

2. Source Code (`one.py`) - Data Loading & Calculations

```
# ============================================================ # STUDENT PERFORMANCE ANALYZER (one.py) # ============================================================ import pandas as pd import numpy as np import matplotlib.pyplot as plt import os def load_student_data(filename="students.csv"): if not os.path.exists(filename): print(f"Error: {filename} file not found.") return pd.DataFrame() try: raw_data = pd.read_csv(filename, header=None, dtype=str) except Exception as error: print("Error reading CSV:", error) return pd.DataFrame() valid_records = [] for _, row in raw_data.iterrows(): if len(row) < 4: continue name = str(row.iloc[0]).strip() marks = [] invalid = False for col in [1, 2, 3]: try: mark = float(str(row.iloc[col]).strip()) if 0 <= mark <= 100: marks.append(mark) else: invalid = True; break except: invalid = True; break if not invalid: valid_records.append([name, marks[0], marks[1], marks[2]]) return pd.DataFrame(valid_records, columns=["Name", "Maths", "Physics", "Chemistry"]) def calculate_results(students): students["Total"] = students[["Maths", "Physics", "Chemistry"]].sum(axis=1) students["Percentage"] = (students["Total"] / 300) * 100 def get_grade(p): return "A" if p>=90 else "B" if p>=75 else "C" if p>=60 else "D" if p>=40 else "F" students["Grade"] = students["Percentage"].apply(get_grade) return students
```

IMPLEMENTATION CODE

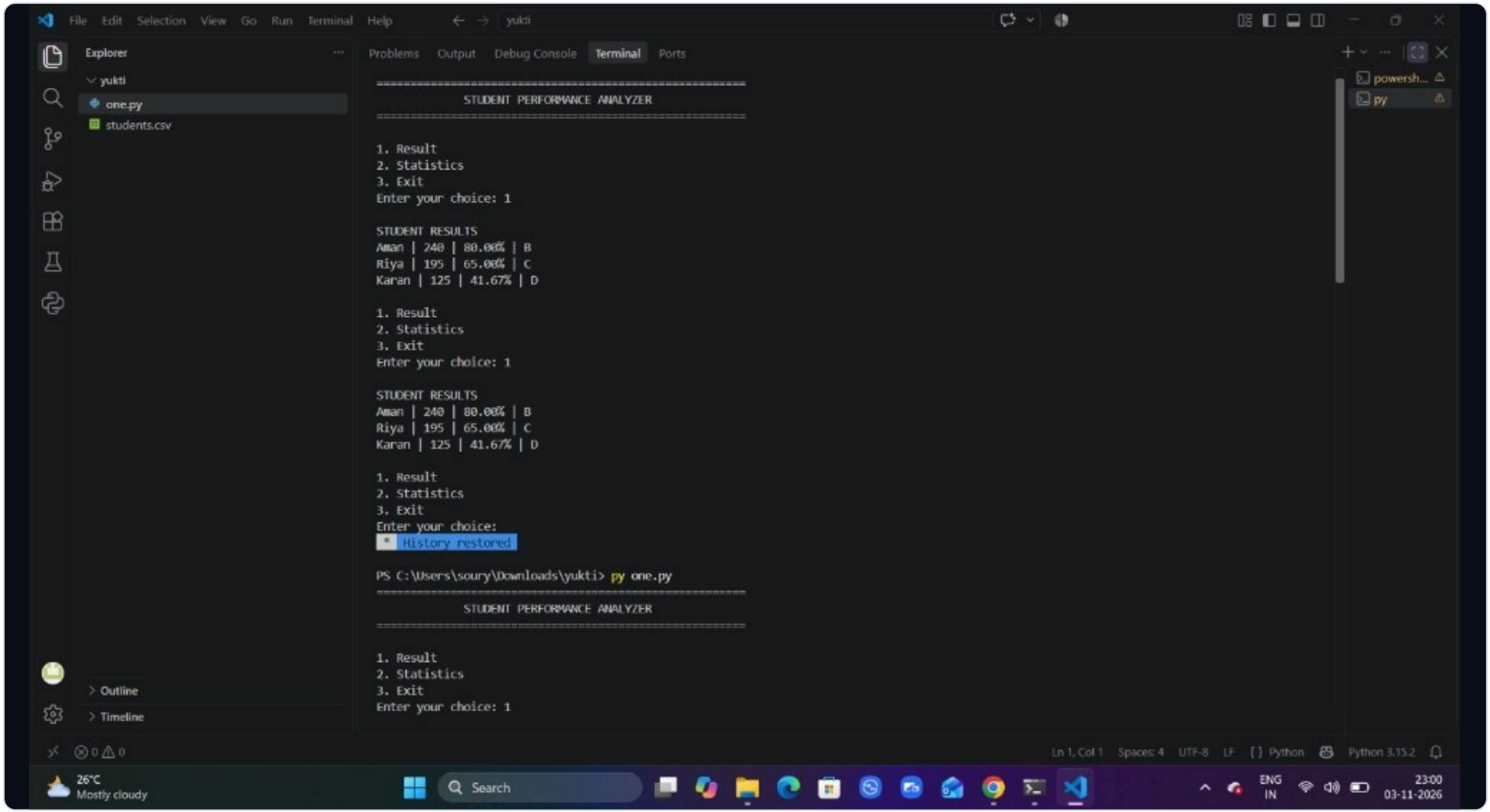
3. Source Code (`one.py`) - Statistics & Visualization

```
def display_statistics(students): percentages = students["Percentage"] mean_percentage = np.mean(percentages) median_percentage = np.median(percentages) std_deviation = np.std(percentages, ddof=0) topper = students.loc[students["Percentage"].idxmax()] subject_averages = students[["Maths", "Physics", "Chemistry"]].mean() print("\nCLASS STATISTICS") print(f"Mean %: {mean_percentage:.2f} | Median %: {median_percentage:.2f} | Std Dev: {std_deviation:.2f}") print(f"Topper: {topper['Name']} ({topper['Percentage']:.2f}%)") print(f"Subject averages: Maths {subject_averages['Maths']:.2f}, Physics {subject_averages['Physics']:.2f}, Chemistry {subject_averages['Chemistry']:.2f}") display_charts(students, subject_averages) def display_charts(students, subject_averages): # Bar Chart plt.figure(figsize=(8, 4.5)) bars = plt.bar(subject_averages.index, subject_averages.values) plt.title("Subject-wise Average Marks") plt.xlabel("Subjects"); plt.ylabel("Average Marks (out of 100)") plt.ylim(0, 110); plt.grid(axis="y", linestyle="--", alpha=0.5) for bar in bars: h = bar.get_height() plt.text(bar.get_x() + bar.get_width()/2, h + 1, f"{h:.2f}", ha="center") plt.tight_layout(); plt.show() # Histogram plt.figure(figsize=(8, 4.5)) plt.hist(students["Percentage"], bins=5, edgecolor="black") plt.title("Student Percentage Distribution") plt.xlabel("Percentage"); plt.ylabel("Number of Students") plt.grid(axis="y", linestyle="--", alpha=0.5) plt.tight_layout(); plt.show()
```

OUTPUT EVIDENCE 01

High Resolution

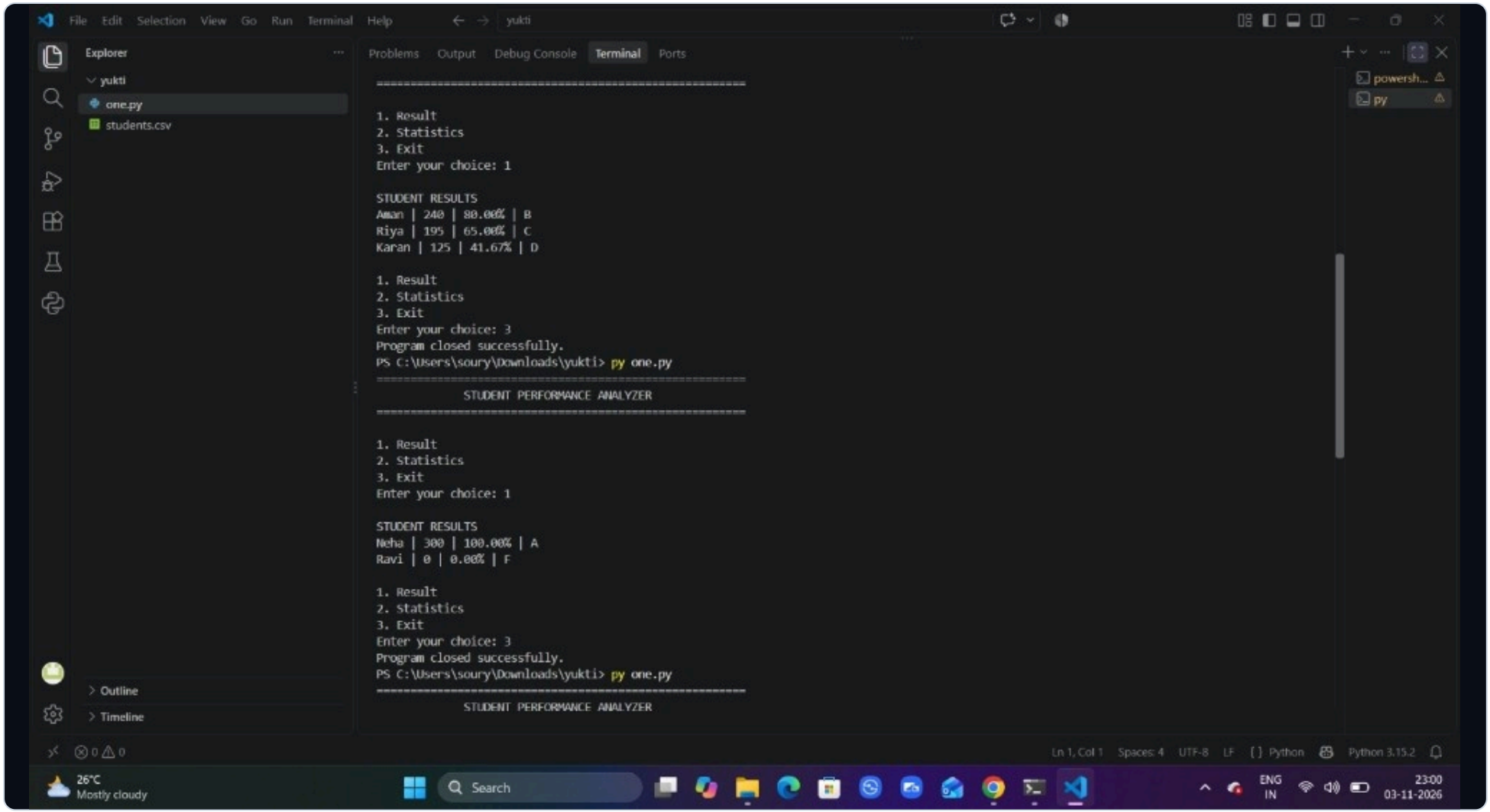
Screenshot 1: Test Case 1 - Student Results Output



OUTPUT EVIDENCE 02

High Resolution

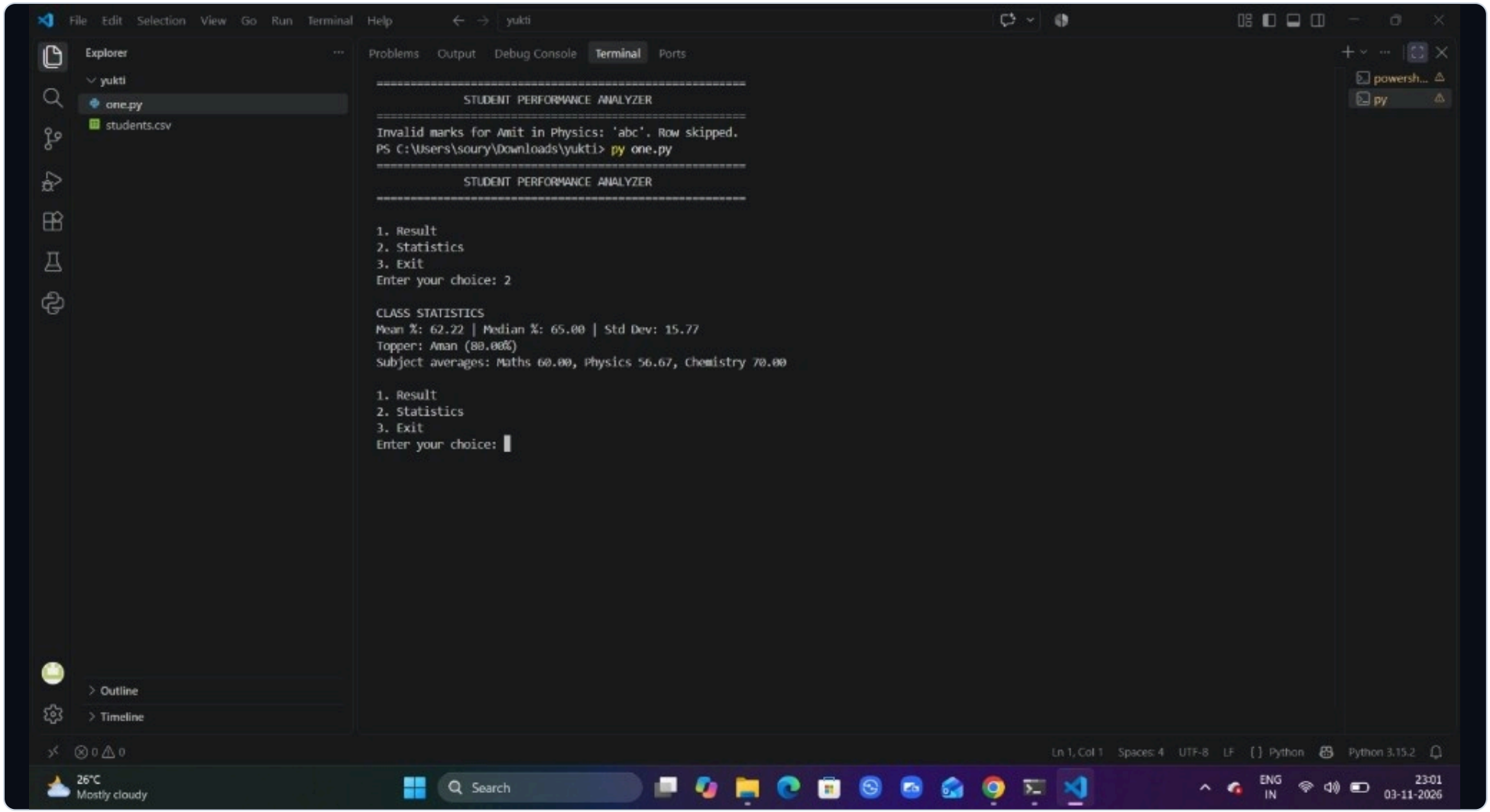
Screenshot 2: Test Case 2 - Minimum / Boundary Results



OUTPUT EVIDENCE 03

Validation Test

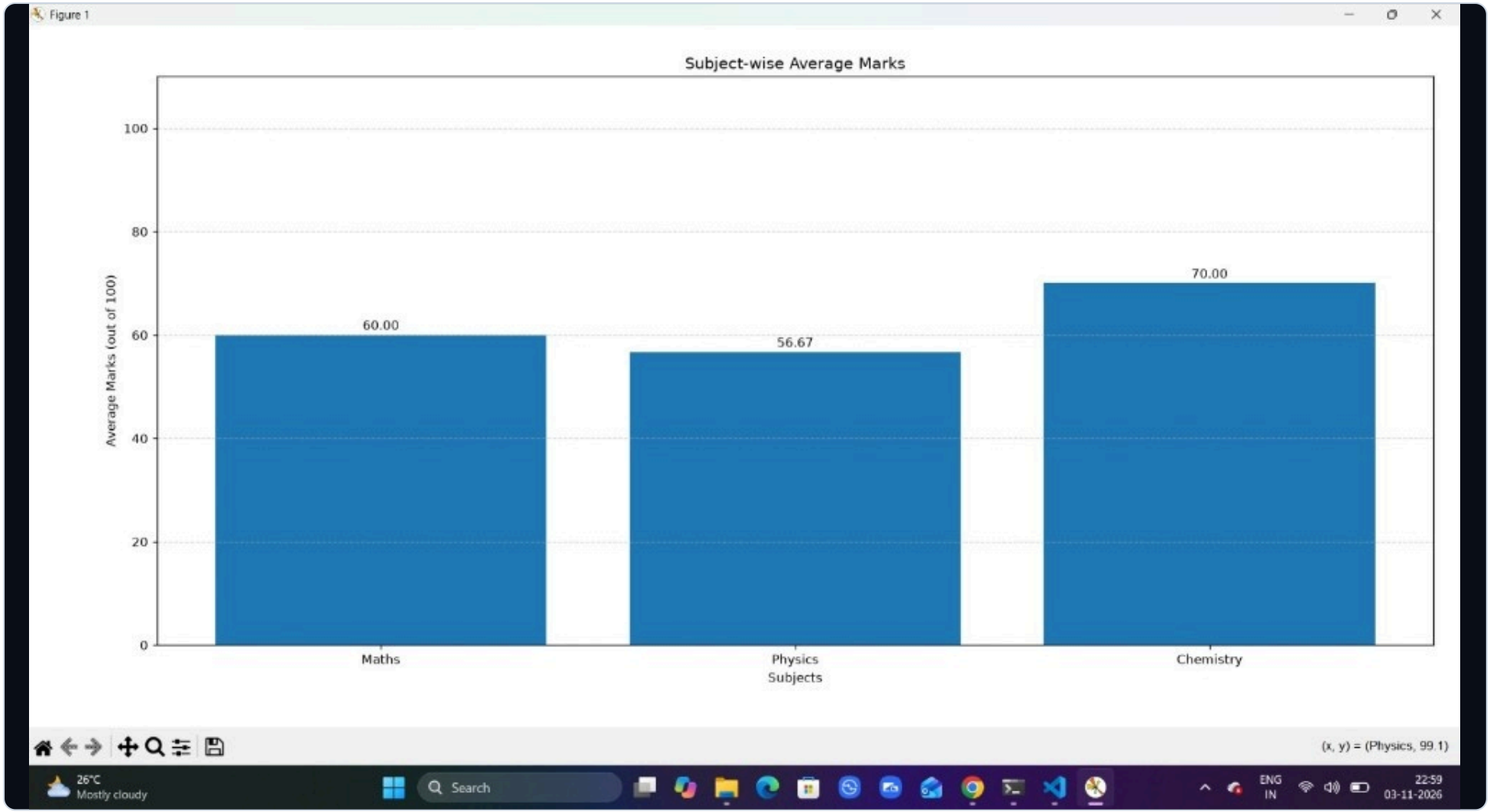
Screenshot 3: Test Case 3 & 4 - Invalid Marks & Class Statistics



OUTPUT EVIDENCE 04

Visualization

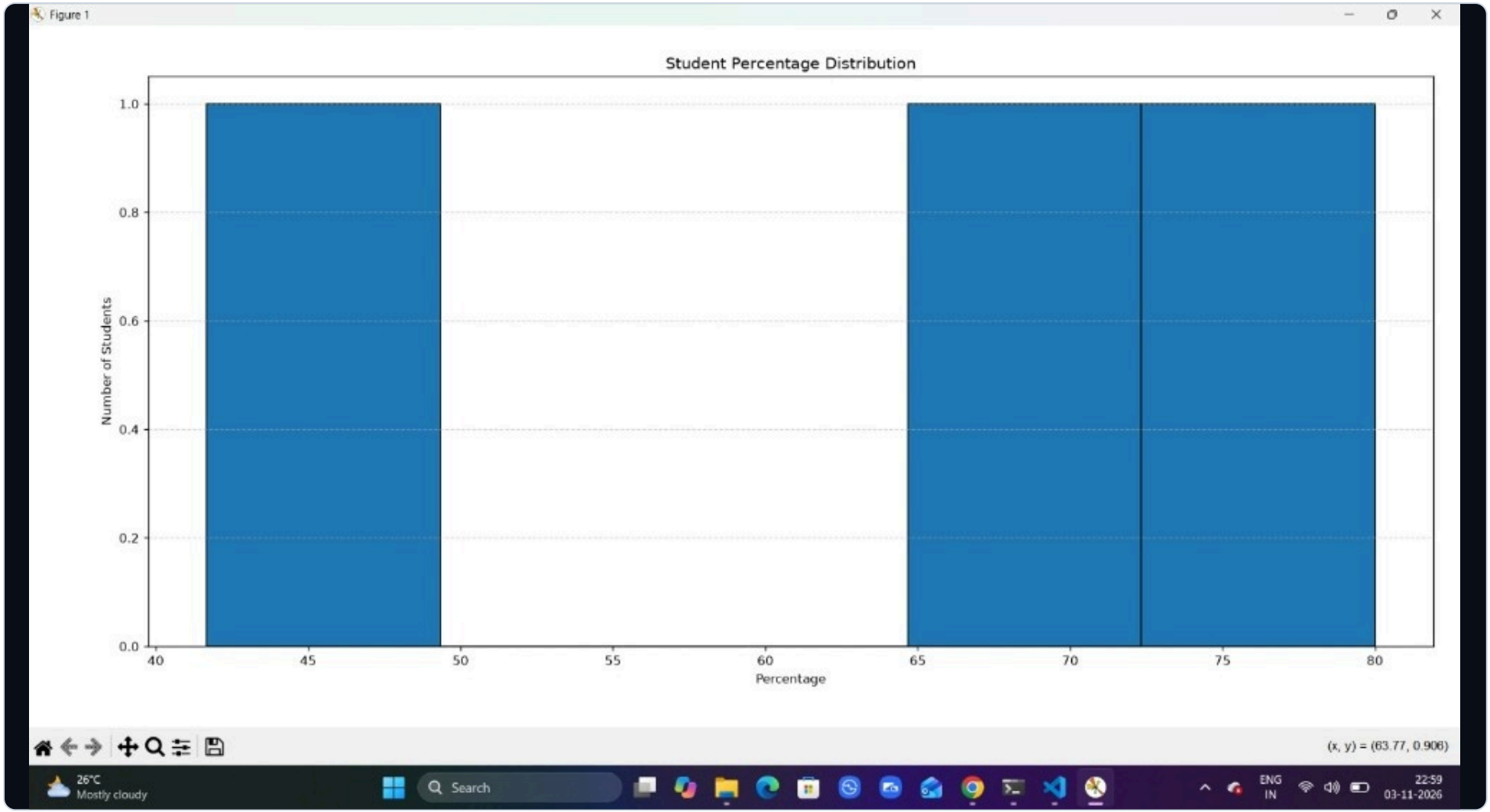
Screenshot 4: Matplotlib Chart - Subject-wise Average Marks



OUTPUT EVIDENCE 05

Visualization

Screenshot 5: Matplotlib Chart - Student Percentage Distribution



VERIFICATION SUMMARY

4. Test Cases & Observed Results

Test Case	Input Data / Action	Observed Result	Status
1. Normal Valid Input	Aman (80,70,90), Riya (60,65,70), Karan (40,35,50)	Displays correct Total, % (80.00%, 65.00%, 41.67%) and Grades (B, C, D).	**PASSED**
2. Boundary Values	Neha (100,100,100), Ravi (0,0,0)	Neha gets 100% (Grade A), Ravi gets 0% (Grade F).	**PASSED**
3. Invalid Mark Handling	Amit (85, 'abc', 70)	Skips invalid row with explicit error notification message.	**PASSED**
4. Class Statistics	Select Option 2 (Statistics)	Mean %: 62.22, Median %: 65.00, Std Dev: 15.77, Topper: Aman.	**PASSED**
5. Matplotlib Charts	Select Option 2 with chart enabled	Renders bar chart for subject averages & percentage histogram.	**PASSED**

Thank You!

The Student Performance Analyzer successfully meets all technical objectives, delivering robust statistical processing and automated graphical analytics.

Yukti Sahu | BCA AI/ML | Rungta International Skills University